

Feeling Terrain Before Crossing: World Models for Off-Road Navigation

E-In Son, Dong-Wook Kim, Ji-Hoon Hwang, Kangsun Lee, Jisung Bae, Jung-Taak Kim and Seung-Woo Seo

Abstract—Navigation world models plan by foresight, predicting the future that each candidate action sequence produces and selecting the best, rather than mapping observations to actions directly. Unlike urban settings where a predicted scene is a sufficient proxy, off-road navigation hinges on the robot-terrain interaction, so the prediction must cover not only what the camera will see but what the robot will feel. However, existing scene-focused models do not predict how much the robot will slip, tilt or shake along a planned trajectory. Proprioception captures these dynamics directly and, when used as input, improves the prediction of the physical future. We present Feel-WM, the first off-road navigation world model that conditions on proprioception and predicts what the robot will feel alongside what the camera will see. The physical future takes the form of a future proprioceptive state and a failure risk, both learned from the robot’s own experience without human labels. The planner rolls out the physical future alongside the scene and weighs the predicted failure risk against goal similarity in a separable score. Experiments on real off-road data and in simulation demonstrate that Feel-WM outperforms visual-only navigation world models in open-loop planning and closed-loop rough-terrain navigation across wheeled and legged platforms. Deployed on a Husky on mountain trails, Feel-WM plans onboard, predicts rough ground ahead and steers around it, completing courses that an end-to-end policy fails.

I. INTRODUCTION

Vision-based navigation has largely relied on end-to-end policies trained on demonstrations to map camera observations to actions [1]–[3]. Navigation world models introduce an intermediate prediction step. Rather than selecting an action directly from the current view, they predict the future of each candidate action sequence and select the sequence whose predicted future is closest to the goal [4]–[7]. Predicting before acting is worth even more on off-road terrain, where an inappropriate action can cause the robot to overturn or become stuck. A robot that crosses a slope safely at low speed can roll over on the same slope when it enters faster with its body already tilted [8], [9].

Yet the future that a navigation world model predicts is what the camera will see. In urban and indoor settings, this view suffices to judge where a plan ends. On off-road terrain, whether a plan succeeds depends on the robot-terrain interaction along the way, and a predicted view captures it only indirectly. The interaction appears in the robot’s own motion, which the model neither observes nor predicts. Recent

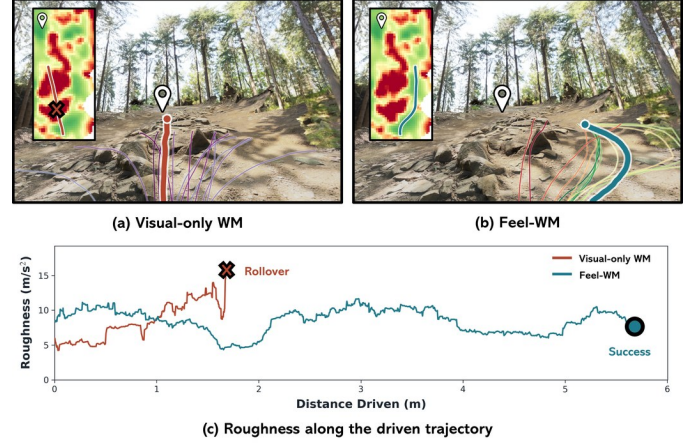


Fig. 1: Same start, goal and terrain, two futures. (a) The visual-only world model drives across the rock field; (b) Feel-WM goes around. Thin lines denote the candidates and the thick line the driven trajectory, which the insets place on a roughness map; $\times$ marks the rollover. (c) Roughness along each driven trajectory.

navigation world models widen what they predict, from how nearby people will move [10] to a terrain map for off-road driving [11], but the robot stays outside.

Proprioception brings the robot into the prediction. An IMU and wheel or joint encoders measure the robot’s own motion, its state and how much it slipped, tilted and vibrated. Several lines of off-road work exploit these signals: forward dynamics models predict the response of the vehicle from its state [8], self-supervised costmaps learn terrain cost from what the vehicle felt [12], and legged traversability estimators infer terrain properties through foot-terrain interaction [13], [14]. These works motivate an off-road navigation world model that predicts what the robot will feel alongside what the camera will see.

We present Feel-WM, the first off-road navigation world model to take proprioception as input and predict the physical future—what the robot will feel before crossing the ground ahead. A proprioceptive encoder conditions the dynamics model on the robot’s current state. Two heads predict the robot’s future proprioceptive state and a failure risk. Since the prediction depends on the robot state and the candidate actions, the model distinguishes a safe crossing from a dangerous one on the same ground. On recorded off-road driving, the

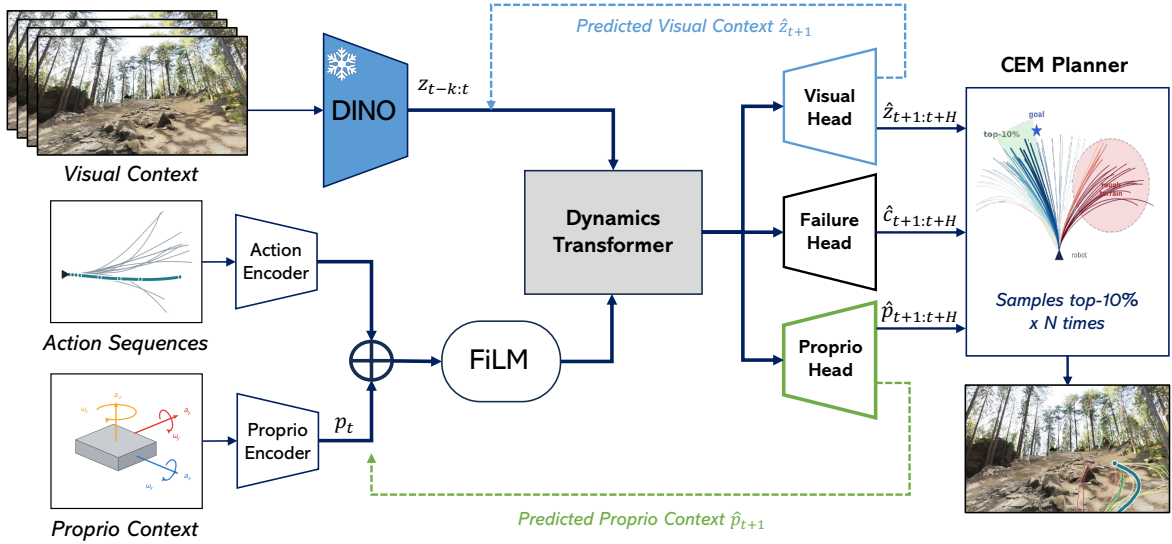


Fig. 2: **Overview of Feel-WM.** FiLM injects the action and the current proprioceptive state into the frozen DINOv2 patch tokens, and the dynamics transformer predicts the next visual tokens, a failure risk and the next proprioceptive state. Dashed: the predictions that re-enter at the next step.

proprioceptive input improves this prediction over the horizons the planner uses. At planning time, the planner rolls out both predictions for each candidate and weighs the predicted failure risk against goal similarity in a score with two separable terms (Fig. 1).

Feel-WM outperforms visual-only navigation world models in open-loop planning on recorded off-road driving and in closed-loop navigation with simulated wheeled and legged robots on rough terrain. Deployed on a Clearpath Husky, Feel-WM plans on an onboard Jetson Orin and completes off-road courses that an end-to-end policy fails.

II. RELATED WORK

A. Navigation World Models

Visual navigation policies are trained end-to-end on demonstrations to map camera observations to actions. ViNT [1] trained a Transformer policy on diverse multi-robot data for image-goal navigation, and NoMaD [2] extended it with a diffusion decoder that represents multiple plausible actions in a scene. Navigation world models instead predict the future of each candidate action sequence and select the best. NWM [4] introduced this paradigm by generating future egocentric views with a conditional diffusion transformer and ranking candidates by the similarity between predicted views and the goal image. Subsequent work has pursued two directions: reducing rollout costs and expanding prediction beyond future visual observations.

From pixels to features. AstraNav-World [15] stayed in pixel space, coupling a video diffusion model to a policy. Generating frames is expensive, however, and later models have traded fidelity for speed. A one-step world model [16] predicts the next observation in a single pass, and DINO-WM [7] ran the rollout over frozen DINOv2 features and

planned toward goal features without task-specific training. ReL-NWM [5] and RAE-NWM [6] followed with a compact latent and a dense representation. Although these models differ in how they represent the future, what they predict remains the scene. DINO-WM concatenates the agent’s proprioceptive state to the latent when one is available, but as the agent’s configuration in tabletop and maze tasks, not as a measurement of how the terrain acts on the robot.

Task-aligned outputs. Each model extends prediction to suit its target setting. NavThinker [10] predicts nearby pedestrians’ trajectories for social navigation, and WMNav [17] uses a vision-language model to guide rollouts toward an object goal. World-action models [18], [19] incorporate actions into the prediction and eliminate the need for a separate planner. TerrainFormer [11] predicts a bird’s-eye-view map of traversability and elevation for off-road driving. These works extend navigation world models beyond future visual observations, but none conditions on the robot’s proprioceptive state or predicts its physical future.

B. Proprioception in Off-Road Navigation

On rough terrain, the effect of an action depends on the robot’s current state as much as on the terrain ahead. Roth et al. [8] condition a forward dynamics model on proprioceptive history and surrounding geometry to predict a legged robot’s future position and failure probability. A sampling-based planner then ranks candidate trajectories by the predicted failure probability rather than a hand-tuned cost. Lee et al. [20] carry the same conditioning to wheeled platforms, predicting position, orientation and bumpiness from terrain and vehicle state. Proprioception also provides supervision: Castro et al. [12] learn an off-road costmap from recorded vehicle vibrations, and ProNav [14] estimates the traversability of the terrain

under a legged robot from joint and IMU signals alone. The forward dynamics models predict the robot's physical future from a geometric terrain representation toward a position goal. Feel-WM predicts it from camera features toward an image goal, alongside the future view.

C. Traversability Estimation and Risk-Aware Planning

In off-road navigation, traversability estimation helps planners identify failures by assigning terrain-dependent costs. Frey et al. [13] estimate traversability online from images, supervised by the robot's own driving, and Velociraptor [21] produces cost and speed maps without human labels. TAO [22] jointly optimizes a path and a speed profile over traversability maps in mountainous terrain. More recently, MAT [9] models traversability as a function of the crossing velocity. For example, a ditch stops a slow vehicle but can be crossed with an accelerated jump. We build on the same premise: the same ground can be passable in one robot state and impassable in another. However, this velocity-conditioned map does not account for the robot's broader proprioceptive state or how it evolves during a traversal.

These lines of research address two complementary requirements for rough-terrain planning: predicting the future scene and the robot's physical future. Feel-WM brings them together. To the best of our knowledge, it is the first world model for off-road navigation that conditions on proprioception, predicts the physical future together with the scene, and plans over both.

III. METHODS

A. Problem Setup and Overview

We consider goal-conditioned navigation on rough terrain. Given a goal image, the robot must select actions that reach the goal without failing along the way.

A navigation world model plans by predicting future observations [4], [5]. A frozen encoder maps each camera frame to a feature map z_t , and a world model W predicts future features conditioned on a candidate action sequence:

$$\hat{z}_{t+1:t+H} = W(z_{t-k:t}, a_{t:t+H-1}), \quad (1)$$

where $a_t = (\Delta x, \Delta y, \Delta \psi, \Delta t)$ specifies a body-frame pose change over a duration Δt , measured from the recorded poses in training; $k+1$ is the number of context frames and H is the rollout horizon. The planner ranks candidate sequences by the similarity between the predicted future features and the goal image features.

On rough terrain, what an action does to the robot depends on the robot's current state as much as on the terrain ahead [8]. However, Eq. (1) neither conditions on the robot's proprioceptive state nor predicts its evolution. Feel-WM extends both the inputs and the outputs:

$$(\hat{z}, \hat{p}, \hat{c})_{t+1:t+H} = W(z_{t-k:t}, p_{t-k:t}, a_{t:t+H-1}), \quad (2)$$

where p_t summarizes the robot's proprioceptive measurements over a window ending at frame t . Since the model reads the recent proprioceptive history, it predicts different futures for

the same ground depending on the robot's state, for example a slope entered slowly or at speed. Alongside future visual features, the model predicts proprioceptive features $\hat{p}$ that describe body vibration, rotation and wheel slip. The model also predicts a failure risk $\hat{c}$, defined as the probability of failure during each rollout step. Fig. 2 provides an overview.

B. Feel-WM

Feel-WM operates entirely in feature space. A frozen DINOv2 encoder [23] maps each frame to patch tokens, which serve as the visual representation for prediction and planning without image decoding.

a) *Reading the robot's state:* Proprioceptive measurements arrive at a much higher rate than camera frames, and their temporal evolution captures motion patterns that a single sample cannot. A small 1-D convolutional network processes the high-rate measurement window ending at frame t (inertial accelerations, angular rates, body velocities, attitude and actuator readings) and pools it into a single embedding p_t . The step's action and p_t condition the current frame's patch tokens through feature-wise linear modulation (FiLM [24]).

b) *Predicting from a shared state:* A dynamics transformer with temporally causal attention over the context tokens produces a shared representation h_t at each step, which feeds three prediction heads. The visual head regresses the next frame's features for each patch, as in a visual-only model. The proprioceptive and failure-risk heads first pool h_t across patches, because what the robot will undergo is a property of the whole view rather than of one patch. The failure risk therefore describes a crossing conditioned on the robot's state and action, rather than a fixed property of a location.

c) *Rolling forward:* An H -step rollout applies this process autoregressively. Neither future observation is available when the rollout is planned, so both streams are advanced from the model's own predictions: predicted patch features are appended to the visual context, and the predicted proprioceptive state is projected back into the conditioning input. After the first step, the model therefore uses its own predictions to advance the rollout.

C. Training

Feel-WM is trained on the robot's own driving. Proprioceptive targets and failure labels are computed from the recordings themselves, without human annotation.

a) *Defining the physical future:* The proprioceptive targets are predefined statistics computed over a short window centered on the target time:

$$\begin{aligned} f_{\text{rough}} &= \sigma(a_z), & f_{\text{ang}} &= \sqrt{\sigma^2(\omega_\phi) + \sigma^2(\omega_\theta)}, \\ f_{\text{slip}} &= \frac{\bar{v}_w - \bar{v}_g}{\max(\bar{v}_w, \bar{v}_g, \epsilon)}, & f_{\text{slope}} &= \overline{v_g \sin \theta}, \end{aligned} \quad (3)$$

where a_z is vertical acceleration, ω_ϕ and ω_θ are roll and pitch rates, v_w is the encoder-derived wheel speed, v_g is the ground speed from the state estimator and θ is pitch; σ and σ^2 are the standard deviation and variance over the window

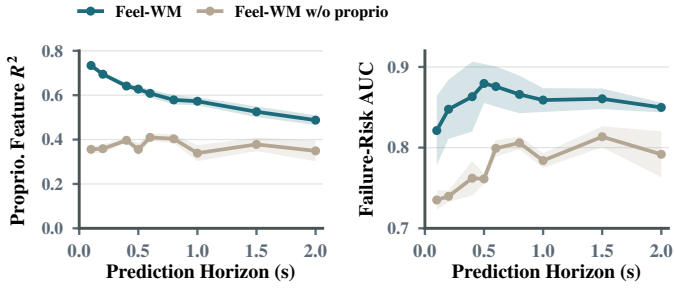


Fig. 3: **Predicting the physical future with and without proprioceptive input.** TartanDrive 2 validation, three seeds, mean $\pm$ std. Left: R^2 of the predicted proprioceptive features. Right: failure-risk AUC.

and overlines denote window means. Roughness and angular instability quantify vertical vibration and variability in roll and pitch rates, respectively. The slip feature measures the relative discrepancy between wheel and ground speeds, and the slope feature serves as a proxy for vertical velocity. All four channels are standardized using z-scores to reduce differences in scale during regression. Binary failure labels are derived from the same recordings: in simulation, a step is labeled unsafe within a predefined interval before a collision, rollover or stall; on TartanDrive 2, when the vehicle’s recorded hazard signal exceeds a fixed threshold.

b) Objective: All three losses are optimized jointly from the start of training. The visual loss combines squared error and cosine distance between predicted and target patch features. The proprioceptive loss applies weighted regression to $\hat{p}$, and the failure-risk loss applies class-weighted cross-entropy to $\hat{c}$ to account for the rarity of unsafe steps. The total objective combines these losses with fixed weights:

$$\mathcal{L} = \mathcal{L}_{\text{vis}} + \lambda_p \mathcal{L}_{\text{prop}} + \lambda_c \mathcal{L}_{\text{fail}}. \quad (4)$$

c) Training the rollout used by the planner: We supervise multi-step rollouts using scheduled sampling. To reflect planning-time operation, where the model recursively consumes its own predictions, we make a single teacher-forcing decision per training sample rather than per step. Each sample therefore uses either ground-truth inputs or its own predictions throughout the rollout. The proportion of autoregressive samples increases as the teacher-forcing probability decays during training.

We make two additional choices to account for the temporal properties of the signals. Because the physical channels decorrelate within seconds, a shared mask restricts both physical losses to steps within a supervision horizon H_p , while the visual loss applies at every step. Each sample also uses a single sampled step duration Δt , with action increments rescaled to preserve their rates. An H -step rollout therefore spans a multi-second lookahead in few recursive predictions, which limits the accumulation of error.

Method	TD1		TD2	
	ADE $\downarrow$	RTE $\downarrow$	ADE $\downarrow$	RTE $\downarrow$
NoMaD [2]	<u>2.226</u>	<u>0.582</u>	3.124	0.755
NWM CDiT-XL [4]	5.356	1.245	5.886	1.347
RAE-NWM B/2 [6]	5.526	1.294	5.877	1.391
Feel-WM	1.666	0.551	<u>3.439</u>	<u>1.016</u>

TABLE I: **Open-loop planning on TartanDrive.** 100 paired planning queries per validation split, 3.2 s horizon; ADE and RTE in meters. Bold: best per column; underline: second best.

D. Planning

The planner searches for an action sequence using the cross-entropy method [25]. At each iteration, it samples candidate sequences, rolls them out through W , scores the predicted futures and refits the sampling distribution to the highest-scoring fraction of candidates. The search optimizes the pose changes in a ; each step’s duration Δt stays fixed.

The score balances goal similarity against predicted failure risk:

$$S(\tau) = \tilde{s}_{\text{goal}} - \lambda_{\text{fail}} \tilde{r}_{\text{fail}}, \quad (5)$$

where each tilde denotes standardization across the candidate pool by subtracting the mean and dividing by the standard deviation. λ_{fail} is therefore a dimensionless trade-off parameter. The goal term s_{goal} is the mean per-patch cosine similarity between the predicted features at the rollout endpoint and the goal image features, matching the objective used by a visual-only world model. The risk term r_{fail} is the mean predicted failure risk $\hat{c}$ over the rollout. Predicted proprioceptive features condition subsequent rollout steps rather than entering the score directly.

We retain separate goal and risk terms because the recordings show what the robot underwent on a given line but not how much risk a mission should accept for progress. The parameter λ_{fail} controls this trade-off and can be adjusted without retraining W .

E. Implementation Details

We use a frozen DINOv2 ViT-B/14 [23] encoder that produces 256 patch tokens per frame, each with 768 dimensions. The proprioceptive encoder is a 1-D convolutional network that processes 50 samples from a causal 0.5 s window ending at each frame and outputs a 128-dimensional embedding. The dynamics transformer has six layers, eight attention heads, an MLP ratio of four and a dropout rate of 0.1. We set $k = 3$, corresponding to four context frames, a rollout horizon of $H = 8$ steps and a supervision horizon of $H_p = 1$ s for the physical losses. Each training sample uses a single step duration drawn from $\{0.1, 0.2, 0.5, 1.0\}$ s. Each proprioceptive target channel is computed over a 0.5 s window centered on the target time.

We set $\lambda_p = 0.5$ and $\lambda_c = 1.0$, with a weight of 0.1 for the cosine term in the visual loss. The teacher-forcing probability remains at 1 for the first quarter of training, decays to 0.5 over the next half and remains at 0.5 for the final quarter. We



Fig. 4: **Qualitative comparison in Gazebo.** Pins and $\times$ denote the goal and rollover. Trajectories are color-coded as visual-only model (red) and Feel-WM (blue).

Environment	Route	Method	Task performance		Ride quality	
			SR (%) $\uparrow$	Prog. (%) $\uparrow$	Rough. (m/s ²) $\downarrow$	Yaw vib. (rad/s) $\downarrow$
Gazebo	Park	Visual-only WM	100.0	99.5	3.83	0.50
		Feel-WM w/o prop.	98.0	99.3	3.74	0.49
		Feel-WM	100.0	99.4	3.71	0.48
	Forest	Visual-only WM	54.0	80.3	4.08	0.52
		Feel-WM w/o prop.	74.0	90.3	4.07	0.74
		Feel-WM	84.0	95.0	4.00	0.52
	Hill	Visual-only WM	58.0	74.5	3.75	0.52
		Feel-WM w/o prop.	68.0	82.9	3.62	0.45
		Feel-WM	84.0	88.6	3.53	0.41
Isaac Sim	Smooth	Visual-only WM	89.7	90.7	5.54	0.45
		Feel-WM w/o prop.	94.6	90.6	5.68	0.45
		Feel-WM	98.9	91.3	5.62	0.40
	Rough	Visual-only WM	67.7	72.4	6.46	0.73
		Feel-WM w/o prop.	62.0	68.9	6.53	0.77
		Feel-WM	78.3	77.4	6.51	0.65

TABLE II: **Closed-loop navigation on rough terrain in two simulators.** 50 episodes per model per route; 3 Gazebo routes and 13 Isaac Sim environments split by route slope into 7 smooth and 6 rough. Bold: best per block.

train with AdamW for 20 epochs using a batch size of 64, a learning rate of 6×10^{-4} and weight decay of 0.05. The learning-rate schedule uses one warm-up epoch followed by cosine decay. The planner refits its sampling distribution to the top 10 % of candidates in each pool and uses the mean of the final elite set for execution.

IV. EXPERIMENTS

Our experiments address three questions. First, does proprioceptive input improve the prediction of the physical future, and do these improvements translate into better plans (Sec. IV-B)? Second, does Feel-WM outperform visual-only navigation world models in closed-loop rough-terrain navigation across wheeled and legged platforms (Sec. IV-C)? Third, can Feel-WM operate on a real robot with onboard planning in the field (Sec. IV-D)?

A. Experimental Setup

a) Robots and data: Our closed-loop experiments use a wheeled Clearpath Husky, except for the legged experiments, which use an ANYmal-C. For offline evaluation, we use TartanDrive 1 [26] and TartanDrive 2 [27], both of which contain off-road driving data collected with an all-terrain vehicle. We use these datasets because they record the wheel

and inertial signals our targets require alongside the camera. Closed-loop simulation experiments use Gazebo and Isaac Sim. We train separate Feel-WM models for each domain using the same architecture and training procedure: one for each TartanDrive dataset, one for Gazebo and one using the legged robot’s walking data. For real-world deployment, we fine-tune the Gazebo model on the real robot’s own driving.

b) Evaluation metrics: We report R^2 for proprioceptive feature prediction and AUC for failure-risk prediction using the recorded hazard labels. For offline planning, we report average displacement error (ADE) and relative trajectory error (RTE). For closed-loop navigation, we report success rate (SR), route progress and two ride-quality metrics: realized roughness, measured as the standard deviation of vertical acceleration, and yaw vibration, measured as the mean absolute heading rate computed from logged poses. Both show whether a model’s success came with more vibration or more heading change.

B. Offline Evaluation on TartanDrive

a) Physical-future prediction: To isolate the effect of the proprioceptive input, we train Feel-WM and Feel-WM w/o proprio, which keeps the heads and drops the input, on TartanDrive 2 and evaluate both on its validation split with

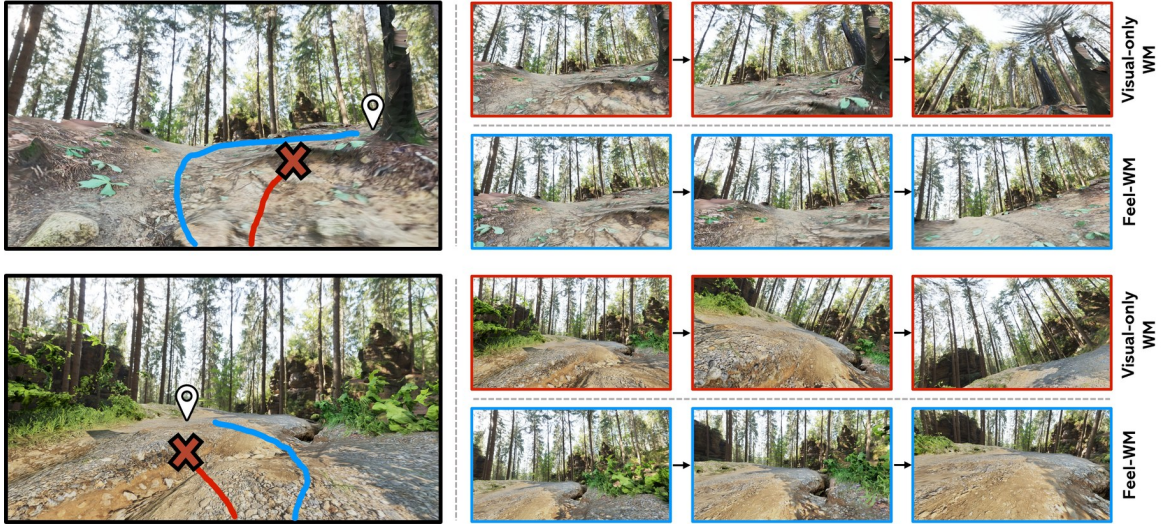


Fig. 5: **Qualitative comparison in Isaac Sim.** Left: trajectories color-coded as visual-only model (red) and Feel-WM (blue); the pin denotes the goal, and $\times$ marks the rollover (top) and the body disturbance on exit (bottom). Right: frames along each episode.

three seeds (Fig. 3). Proprioceptive input raises the R^2 of the predicted features and the AUC of the failure risk at every horizon shown, and the gain holds on every seed.

b) Open-loop planning: We compare against released checkpoints of three baselines: NWM [4] and RAE-NWM [6], which predict future visual observations, and NoMaD [2], a diffusion policy that directly generates actions. For open-loop planning we use the TartanDrive 1 model of Feel-WM on both datasets, so that Feel-WM, NWM and NoMaD were all trained on TartanDrive 1, RAE-NWM on other navigation datasets, and no model was trained on TartanDrive 2. Following NWM’s protocol, world-model planning selects candidate action sequences based on the similarity between predicted future observations and the goal image. All world models use the same CEM configuration and action prior over a 3.2 s horizon. We evaluate the resulting plans against recorded driver trajectories (Table I).

Among the world models, Feel-WM achieves the lowest ADE on both datasets. NoMaD achieves a lower ADE on TartanDrive 2. This metric measures agreement with the recorded driver trajectory and therefore aligns closely with NoMaD’s imitation objective. TartanDrive 1 records no failure labels, so its model omits the failure-risk head and loss while keeping the proprioceptive input and head; all world models here therefore plan on goal similarity alone. No plan here is executed, so this evaluation does not assess the navigation benefit of the failure-risk term.

C. Closed-Loop Navigation in Simulation

We evaluate in two simulators. In Gazebo, we use the natural outdoor environments of NEGS-UGV [28], with three routes spanning 110–165 m through a flat park, an uneven forest and a steep, uneven hill. In Isaac Sim, we use the outdoor environments of TAO [22], reconstructed from real

campus paths and mountain trails: 13 environments with routes of 8–30 m, divided by route slope into 7 smooth and 6 rough. All models are trained on 880 Gazebo trajectories totaling 14.0 h, with no training on Isaac Sim. Evaluation in Isaac Sim therefore tests zero-shot transfer.

a) Compared models: We compare three models with a shared backbone architecture, training procedure, and training duration. The visual-only world model conditions on camera observations and predicts future visual features. Feel-WM additionally conditions on recent proprioceptive measurements and predicts future proprioceptive features and failure risk. Feel-WM w/o proprio keeps these heads and drops the proprioceptive input.

NWM and RAE-NWM were not trained in either simulator, and their diffusion planners cost minutes per plan, or about a day of GPU time per episode at our replanning rate. We therefore use a visual-only world model trained on the same corpus as the controlled baseline for that paradigm.

b) Driving protocol: Each model runs 50 episodes per route, following goal images recorded during a single human-driven traversal. An episode is successful if the robot reaches within 1 m of the final goal. The planner evaluates 64 candidates per CEM iteration over two iterations and replans every four control steps. Its eight-step rollout spans 3.4 s with step durations of 0.1, 0.5 and 1.0 s. The visual-only model plans on goal similarity alone, since it has no failure-risk head. Feel-WM and Feel-WM w/o proprio use Eq. (5) with $\lambda_{\text{fail}} = 1$.

c) Gazebo routes: On the flat park route, all three models reach the goal in nearly every episode (Table II): flat ground gives the physical future nothing to add. Performance differences emerge on more challenging terrain. On the uneven forest route, Feel-WM achieves an SR of 84.0%, compared with 54.0% for the visual-only model and 74.0% for Feel-WM w/o proprio. On the steep, uneven hill route, Feel-WM

Route	Method	SR	Prog.	Rollover
Rough	Visual-only WM	31.0	43.7	51.0
	Feel-WM	44.0	46.2	37.0
Smooth	Visual-only WM	52.5	75.3	33.3
	Feel-WM	78.3	77.7	12.5

TABLE III: **Closed-loop navigation on a legged robot.** We evaluate an ANYmal-C in Isaac Sim over 6 smooth and 5 rough environments, 20 episodes each. All columns in %.

succeeds in 84.0 % against 58.0 and 68.0 %. The comparison between Feel-WM and Feel-WM w/o proprio isolates the contribution of proprioceptive conditioning. Feel-WM also achieves the lowest realized roughness on all three routes and the lowest yaw vibration on the park and hill routes, indicating that its higher success rate does not come at the expense of ride quality.

Fig. 4 presents paired episodes for each route. The trajectories nearly coincide in the park. In the forest and on the hill, the visual-only model maintains its course through rough terrain and rolls over, whereas Feel-WM detours around it.

d) *Zero-shot in Isaac Sim:* In Isaac Sim, Feel-WM achieves the highest SR on rough terrain at 78.3 %, compared with 67.7 % for the visual-only model and 62.0 % for Feel-WM w/o proprio. On smooth terrain all three models succeed in most episodes, with Feel-WM highest.

Fig. 5 shows paired episodes from two environments. In the first, a large tree root obstructs the direct path to the goal. The visual-only model attempts to cross it and rolls over. Feel-WM predicts high failure risk for this crossing and steers around the root to reach the goal. In the second, an eroded depression runs along the center of the path. Both models succeed, but Feel-WM traverses the shallow edge while the visual-only model crosses the deepest section and undergoes a large body disturbance on exit.

The roughness metric shows little separation between models in Isaac Sim, with the visual-only model achieving a slightly lower value. Feel-WM travels faster over the same terrain, which raises its measured roughness. Feel-WM nevertheless achieves the lowest yaw vibration in both terrain groups.

e) *Score ablation:* Planning the same Feel-WM checkpoint on the hill route with goal similarity alone, Eq. (5) at $\lambda_{\text{fail}} = 0$, lowers SR from 84.0 to 62.0 % and progress from 88.6 to 77.9 % over 50 episodes. Predicted failure risk therefore improves trajectory selection on rough terrain.

f) *Legged navigation:* We evaluate on a legged platform to assess whether the benefits of Feel-WM extend beyond wheeled robots. We train the visual-only world model and Feel-WM on 815 trajectories collected by an ANYmal-C in Isaac Sim. We replace the wheel slip of Eq. (3) with foot slip and add a cost-of-transport channel. The robot walks with the pretrained rough-terrain locomotion policy of Isaac Lab [29], which tracks the body-frame velocity commands that the planner issues; both world models command the same



Fig. 6: **Qualitative comparison on the real robot.** The pin denotes the next goal and the red region the hazard on the direct line. Arrows are color-coded as the trajectory driven by NoMaD (orange) and by Feel-WM (blue).

Method	Roots		Downhill		Uphill	
	SR	Rough.	SR	Rough.	SR	Rough.
NoMaD	60	0.303	30	0.731	50	0.783
Feel-WM	90	0.261	70	0.593	80	0.612

TABLE IV: **Closed-loop navigation on the real robot.** We evaluate a Clearpath Husky on three courses, 10 runs per course and method. SR in %, roughness in m/s².

policy. Two of the 13 held-out environments are excluded as unsuitable for legged locomotion, leaving six smooth and five rough. We evaluate each model over 20 episodes per environment and report rollover rate in place of roughness (Table III). Feel-WM achieves higher success rates in both groups: 44.0 % versus 31.0 % on rough terrain and 78.3 % versus 52.5 % on smooth terrain. It also reduces rollover rates in both groups while maintaining similar progress, indicating that its higher success rate is accompanied by fewer falls.

D. Real-World Deployment

a) *Robot platform:* We deploy Feel-WM on a Clearpath Husky with a ZED 2i stereo camera and an IMU, planning onboard a Jetson AGX Orin. We fine-tune Feel-WM from its Gazebo model and NoMaD from its released checkpoint. Both use under an hour of the Husky’s own driving, recorded on trails away from the evaluation courses.

b) *Onboard planning:* We fine-tune Feel-WM with a perceiver resampler [30] that compresses the 256 patch tokens of each frame to 32 latents before the dynamics transformer and restores the grid after it, so the heads and the rollout keep full resolution while the transformer runs on an eighth of the tokens. Onboard the planner evaluates 16 candidates over two CEM iterations and replans every 0.8 s. The world-model baselines are not deployed: NWM reports 30 s to simulate a single trajectory on a desktop GPU [4], and RAE-NWM rolls out with 50 ODE steps per frame [6], far outside an onboard replanning budget.

c) *Field results:* We drive three courses, 10 runs per course and method (Table IV). NoMaD follows a goal-image graph built from the same pass. A run counts as a success only if it reaches the goal with no operator intervention. Feel-WM completes 24 of the 30 runs against 14 for NoMaD and rides smoother on every course. Fig. 6 shows two of these runs. A rock field lies on the direct line to the next goal; NoMaD holds

that line and drives across it, while Feel-WM passes beside it and rejoins the taught route beyond.

V. CONCLUSION

We presented Feel-WM, a navigation world model that conditions on the robot's proprioceptive state to predict the physical future alongside the scene, together with a planner that weighs the predicted failure risk against goal similarity. Feel-WM outperforms visual-only navigation world models in open-loop planning on recorded off-road driving data and in closed-loop rough-terrain navigation across wheeled and legged robots. Deployed on a Clearpath Husky, it completes off-road courses that an end-to-end policy fails.

Two limitations motivate future work. First, Feel-WM learns the physical future solely from recorded driving data, whose limited volume and diversity constrain generalization to unseen terrain. World models that learn from experience share this dependence on training coverage. One direction is to train a visual encoder on off-road driving data with objectives that capture terrain properties relevant to the robot-terrain interaction. Second, onboard planning constrains model size and rollout computation, creating a trade-off between prediction accuracy and computational cost. Future work on this trade-off includes generative rollouts efficient enough for onboard sampling, more compact frame representations, and models that jointly predict actions and future scenes.

REFERENCES

- [1] D. Shah, A. Sridhar, N. Dashora, K. Stachowicz, K. Black, N. Hirose, and S. Levine, "ViNT: A foundation model for visual navigation," in *Proc. Conf. Robot Learn. (CoRL)*, 2023, pp. 711–733.
- [2] A. Sridhar, D. Shah, C. Glossop, and S. Levine, "NoMaD: Goal masked diffusion policies for navigation and exploration," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024, pp. 63–70.
- [3] D.-W. Kim, J.-H. Hwang, E.-I. Son, M. Oh, and S.-W. Seo, "CanonNav: Disentangling navigation behavior from camera geometry in cross-platform visual navigation," *arXiv:2608.30242*, 2026.
- [4] A. Bar, G. Zhou, D. Tran, T. Darrell, and Y. LeCun, "Navigation world models," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2025, pp. 15 791–15 801.
- [5] Z. Zhang, H. Zhang, K. Huang, C. Shi, and H. Lu, "Efficient image-goal navigation with representative latent world model," *arXiv:2511.11011*, 2025.
- [6] M. Zhang, W. Shen, F. Zhang, H. Qin, Z. Pei, and Z. Meng, "RAE-NWM: Navigation world model in dense visual representation space," *arXiv:2603.09241*, 2026.
- [7] G. Zhou, H. Pan, Y. LeCun, and L. Pinto, "DINO-WM: World models on pre-trained visual features enable zero-shot planning," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2025, pp. 79 115–79 135.
- [8] P. Roth, J. Frey, C. Cadena, and M. Hutter, "Learned perceptive forward dynamics model for safe and platform-aware robotic navigation," in *Proc. Robot.: Sci. Syst. (RSS)*, 2025.
- [9] Z. Zhao, T. Fu, S. Su, Q. Du, E. T. Esfahani, K. Dantu, S. Chowdhury, and C. Wang, "Learning when to jump for off-road navigation," *arXiv:2602.00877*, 2026.
- [10] T. Hu, Z. Gong, L. Kong, X. Mei, Y. Ding, Q. Zeng, A. Liang, R. Li, Y. Zhong, and J. Liang, "NavThinker: Action-conditioned world models for coupled prediction and planning in social navigation," *arXiv:2603.15359*, 2026.
- [11] Y. Yang and K. Ricks, "TerrainFormer: World model-guided decision transformer for autonomous off-road navigation," *Sensors*, vol. 26, no. 12, p. 3795, 2026.
- [12] M. G. Castro, S. Triest, W. Wang, J. M. Gregory, F. Sanchez, J. G. Rogers, and S. Scherer, "How does it feel? Self-supervised costmap learning for off-road vehicle traversability," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2023, pp. 931–938.
- [13] J. Frey, M. Mattamala, N. Chebrolu, C. Cadena, M. Fallon, and M. Hutter, "Fast traversability estimation for wild visual navigation," in *Proc. Robot.: Sci. Syst. (RSS)*, 2023.
- [14] M. Elnoor, A. J. Sathyamoorthy, K. Weerakoon, and D. Manocha, "ProNav: Proprioceptive traversability estimation for legged robot navigation in outdoor environments," *IEEE Robot. Autom. Lett.*, vol. 9, no. 8, pp. 7190–7197, 2024.
- [15] J. Chen, J. Hu, H. Bai, M. Luo, X. Xue, B. Ren, C. Bai, S. Xie, Z. Chen, F. Liu *et al.*, "AstraNav-World: World model for foresight control and consistency," *arXiv:2512.21714*, 2025.
- [16] W. Shen, Z. Meng, J. Ma, M. Zhou, and D. Xiang, "An efficient and multi-modal navigation system with one-step world model," *arXiv:2601.12277*, 2026.
- [17] D. Nie, X. Guo, Y. Duan, R. Zhang, and L. Chen, "WMNav: Integrating vision-language models into world models for object goal navigation," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2025, pp. 2392–2399.
- [18] D. Azuma, T. Miyashita, K. Sakamoto, S. Kurita, Y. Zhu, P. Khrapchenkov, M. Kawanabe, Y. Iwasawa, and Y. Matsuo, "NavWAM: A navigation world action model for goal-conditioned visual navigation," *arXiv:2606.13494*, 2026.
- [19] N. Yang, Y. Huang, K. Peng, Z. He, K. Wang, C. Miao, K. Lyu, G. Li, X. Wang, Z. Zhu *et al.*, "WAM-Nav: Asymmetric latent world-action modeling for unified visual navigation," *arXiv:2606.04907*, 2026.
- [20] J. H. Lee, J. Choi, S. Ryu, H. Oh, S. Choi, and J. Hwangbo, "Learning vehicle dynamics from cropped image patches for robot navigation in unpaved outdoor terrains," *IEEE Robot. Autom. Lett.*, vol. 9, no. 5, pp. 4035–4042, 2024.
- [21] S. Triest, M. Sivaprakasam, S. Aich, D. Fan, W. Wang, and S. Scherer, "Velociraptor: Leveraging visual foundation models for label-free, risk-aware off-road navigation," in *Proc. Conf. Robot Learn. (CoRL)*, 2024.
- [22] S.-W. Yoo, E.-I. Son, and S.-W. Seo, "Traversability-aware adaptive optimization for path planning and control in mountainous terrain," *IEEE Robot. Autom. Lett.*, vol. 9, no. 6, pp. 5078–5085, 2024.
- [23] M. Oquab, T. Darcet, T. Moutakanni, H. V. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, M. Assran, N. Ballas, W. Galuba, R. Howes, P.-Y. Huang, S.-W. Li, I. Misra, M. Rabbat, V. Sharma, G. Synnaeve, H. Xu, H. Jegou, J. Mairal, P. Labatut, A. Joulin, and P. Bojanowski, "DINOv2: Learning robust visual features without supervision," *Trans. Mach. Learn. Res.*, 2024.
- [24] E. Perez, F. Strub, H. De Vries, V. Dumoulin, and A. Courville, "FiLM: Visual reasoning with a general conditioning layer," in *Proc. AAAI Conf. Artif. Intell.*, vol. 32, 2018.
- [25] P.-T. de Boer, D. P. Kroese, S. Mannor, and R. Y. Rubinstein, "A tutorial on the cross-entropy method," *Ann. Oper. Res.*, vol. 134, no. 1, pp. 19–67, 2005.
- [26] S. Triest, M. Sivaprakasam, S. J. Wang, W. Wang, A. M. Johnson, and S. Scherer, "TartanDrive: A large-scale dataset for learning off-road dynamics models," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2022, pp. 2546–2552.
- [27] M. Sivaprakasam, P. Maheshwari, M. G. Castro, S. Triest, M. Nye, S. Willits, A. Saba, W. Wang, and S. Scherer, "TartanDrive 2.0: More modalities and better infrastructure to further self-supervised learning research in off-road driving tasks," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024.
- [28] M. Sánchez, J. Morales, J. L. Martínez, J. J. Fernández-Lozano, and A. García-Cerezo, "Automatically annotated dataset of a ground mobile robot in natural environments via gazebo simulations," *Sensors*, vol. 22, no. 15, p. 5599, 2022.
- [29] M. Mittal, C. Yu, Q. Yu, J. Liu, N. Rudin, D. Hoeller, J. L. Yuan, R. Singh, Y. Guo, H. Mazhar, A. Mandlekar, B. Babich, G. State, M. Hutter, and A. Garg, "Orbit: A unified simulation framework for interactive robot learning environments," *IEEE Robot. Autom. Lett.*, vol. 8, no. 6, pp. 3740–3747, 2023.
- [30] J.-B. Alayrac, J. Donahue, P. Luc, A. Miech, I. Barr, Y. Hasson, K. Lenc, A. Mensch, K. Millican, M. Reynolds *et al.*, "Flamingo: a visual language model for few-shot learning," *Adv. Neural Inf. Process. Syst.*, vol. 35, pp. 23 716–23 736, 2022.